Introduction to Unsupervised Learning

Finding hidden patterns without labels — Clustering · Dimensionality Reduction · Anomaly Detection

K-Means | DBSCAN | PCA | t-SNE | Autoencoders | Isolation Forest

AI MASTER CLASS

What We'll Cover Today

01
Supervised vs Unsupervised
The fundamental difference & when to choose

02
Clustering
K-Means · DBSCAN · Hierarchical · Evaluation

03
Dimensionality Reduction
PCA · t-SNE · UMAP · Autoencoders

04
Anomaly Detection
Isolation Forest · One-Class SVM · LOF

05
Association Rules
Apriori · FP-Growth · Market basket analysis

06
Real-World Applications
When & how to use unsupervised learning

Supervised vs Unsupervised Learning — The Key Difference

SUPERVISED Learning	
Has labels?	YES — every sample has a correct answer
Goal	Learn a mapping from input → known output
What it learns	A decision boundary or regression surface
Output	Predicts class labels or numeric values
Examples	Email spam, house price, disease diagnosis
Algorithms	Linear Reg, Logistic Reg, RF, XGBoost, SVM
Needs	Large labeled dataset — expensive to collect

UNSUPERVISED Learning	
Has labels?	NO — raw data only, no correct answers
Goal	Discover hidden structure & patterns
What it learns	Natural groupings, features, anomalies
Output	Clusters, compressed representations, scores
Examples	Customer segments, fraud, topic modeling
Algorithms	K-Means, DBSCAN, PCA, t-SNE, Autoencoder
Needs	Only raw data — much easier to collect

Why Unsupervised Learning? The Real-World Problem

The Hard Truth: In the real world, 99% of data has NO labels. Labeling is expensive, slow, and sometimes impossible. Unsupervised learning lets us extract value from raw, unlabeled data — which is everywhere.

No Labels Available

A hospital has 10 million patient records but no doctor has labeled them. Clustering can find patient sub-groups automatically.

Only 0.5%

of real data is labeled

Explore Unknown Data

You receive a new dataset. You don't know its structure. Unsupervised methods reveal what patterns exist BEFORE you build any model.

EDA first

always, always, always

Reduce Complexity

A gene expression dataset has 30,000 features. PCA compresses it to 50 components while keeping 95% of the information.

600× less

dimensions needed

Detect the Unknown

You can't label fraud types you've never seen. Anomaly detection flags anything that looks unusual — no fraud labels needed.

$42B+

fraud prevented yearly by ML

The Landscape of Unsupervised Learning — Full Map

Unsupervised Learning

CLUSTERING

- K-Means — partition into K centroids
- DBSCAN — density-based, finds any shape
- Hierarchical — tree of merges/splits
- Gaussian Mixture — soft probabilistic
- Mean Shift — finds density peaks
- Spectral — graph-based partitioning

DIM. REDUCTION

- PCA — linear variance maximization
- t-SNE — nonlinear visualization (2D/3D)
- UMAP — faster, preserves global struct.
- Apriori — frequent itemsets + rules

ASSOCIATION RULES

- Autoencoders — neural network compress.
- LDA — topic discovery in text
- FP-Growth — faster tree-based mining

ANOMALY DETECTION

- Isolation Forest — isolates outliers
- One-Class SVM — decision boundary
- Local Outlier Factor (LOF) — density
- Autoencoder — reconstruction error
- Elliptic Envelope — Gaussian assumption
- Z-Score / IQR — statistical baseline

01

Clustering

Grouping similar data points — without labels

K-Means Clustering — The Classic Algorithm

Partition n data points into exactly K clusters by iteratively optimizing centroid positions.

Analogy: Imagine students randomly assigned to 3 study groups. Each round, they move to the group whose center is closest. Groups shift until no one wants to switch. That's K-Means.

1 Initialize

Place K centroids randomly in the feature space. (Or use K-Means++ for smarter initialization.)

2 Assign

Assign each data point to the nearest centroid using Euclidean distance: $\text{argmin} \, ||x_i - \mu_j||^2$

3 Update

Recompute each centroid as the mean of all points assigned to it: $\mu_j = (1/|C_j|) \cdot \Sigma_{i \in C_j} x_i$

4 Repeat

Repeat steps 2–3 until centroids stop moving (convergence) or max iterations reached.

Objective Function

Minimize inertia = $\Sigma_j \Sigma_{i \in C_j} ||x_i - \mu_j||^2$
Smaller inertia = tighter, more compact clusters.

Choosing K: Elbow Method

Plot inertia vs K. The 'elbow' where the curve bends is the optimal K. Also use Silhouette Score.

Key Limitations

Assumes spherical clusters. Sensitive to outliers and K choice. Fails on non-convex shapes.

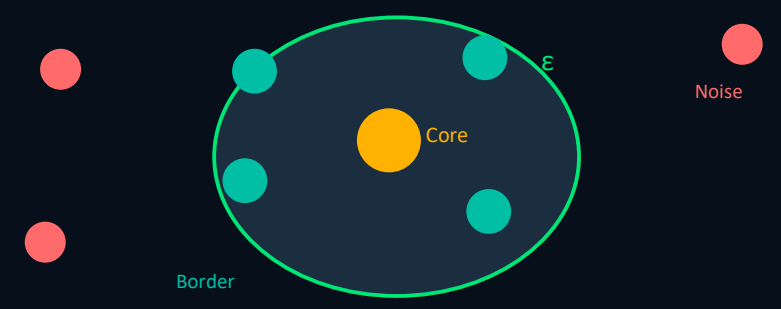
DBSCAN — Density-Based Clustering

Discovers clusters of ANY shape and automatically labels outliers as noise — no K needed.

DBSCAN — Key Concepts

ε (epsilon)	Neighborhood radius. Any point within ε distance is a 'neighbor'. You choose this parameter.
minPts	Minimum number of points within ε to form a dense region. Typical: minPts = 2× dimensions.
Core Point	A point with at least minPts neighbors within ε. The 'dense' center of a cluster.
Border Point	Within ε of a core point but has fewer than minPts neighbors itself. At the cluster edge.
Noise Point	Not a core point and not within ε of any core point. Labeled −1. These are OUTLIERS!
Algorithm	Expand clusters from core points by density-reachability. No centroids, no K needed.

Visual: Point Types in DBSCAN



	K-Means	DBSCAN
Needs K?	YES	NO ✓
Cluster shape	Spherical only	Any shape ✓
Outlier handling	Forces into cluster	Labels as noise ✓
Speed	O(n·K·iter) — fast	O(n log n) — slower
Best for	Compact blobs	Arbitrary shapes

Hierarchical Clustering — Tree of Relationships

Build a dendrogram: a hierarchy of merges/splits. You choose K AFTER seeing the full tree.

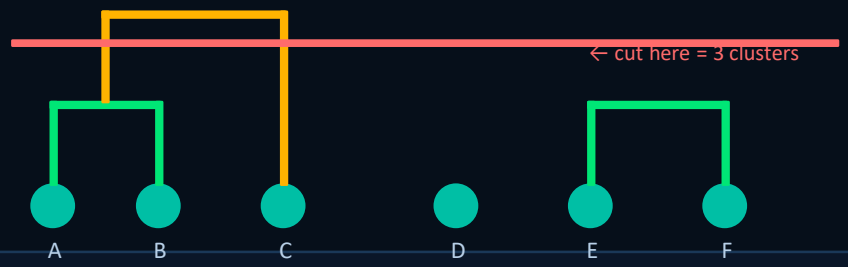
Analogy: Biological taxonomy: species → genus → family → order → class → phylum → kingdom. Each merge captures a level of similarity. The dendrogram IS your model — you cut it wherever you want.

Agglomerative (Bottom-Up) — most common

1. Start: each data point is its own cluster (n clusters)
2. Find the two closest clusters (using linkage method)
3. Merge them into one cluster (n−1 clusters now)
4. Repeat until all points are in one cluster (root)
5. Cut the dendrogram at desired height to get K clusters

Linkage: Single (min dist) · Complete (max dist) · Average · Ward (min variance — most popular)

Dendrogram — cut at height 3 → 3 clusters



Divisive (Top-Down)

Start with one big cluster, recursively split into smaller ones. Less common but useful when you only need top-level groups. DIANA algorithm implements this approach.

How Do We Evaluate Clustering? — No Labels, No Problem

Without ground truth labels, we use internal metrics that measure cohesion (within-cluster tightness) and separation (between-cluster distance).

Silhouette Score

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

−1 to +1 (higher = better)

✓ > 0.5

a(i) = avg distance to same cluster. b(i) = avg distance to nearest other cluster. Score near +1 means well-clustered. Near 0 = on boundary. Near −1 = wrong cluster.

Davies-Bouldin Index

$$DB = (1/K) \cdot \Sigma_k \max_{k \neq l} (\sigma_k + \sigma_l) / d(c_k, c_l)$$

0 to ∞ (lower = better)

✓ < 0.5

Ratio of within-cluster scatter to between-cluster distance. Lower DB = clusters are compact AND well-separated from each other.

Inertia (WCSS)

$$W = \Sigma_k \Sigma_{i \in C_k} ||x_i - \mu_k||^2$$

0 to ∞ (lower = better)

✓ Use Elbow

Sum of squared distances to cluster centroids. Used in the Elbow Method: plot inertia vs K and look for the 'elbow' — where the benefit of adding K diminishes.

Calinski-Harabasz Index

$$CH = [tr(B_k)/(K-1)] / [tr(W_k)/(n-K)]$$

0 to ∞ (higher = better)

✓ Maximize

Ratio of between-cluster to within-cluster dispersion. Higher = more defined clusters. Fast to compute. Works well for convex clusters.

02

Dimensionality

Reduction

The Curse of Dimensionality — Why We Need Dim. Reduction

Key insight: As the number of features (dimensions) grows, data becomes increasingly sparse. Distance metrics lose meaning. Models overfit. Storage explodes. We need to reduce dimensions while keeping the information that matters.

Distance Loses Meaning

In 1000D space, all points are nearly equidistant. KNN breaks down. Clusters become impossible to find.

Storage Explodes

An image with 1000×1000 pixels = 1 million features. Most ML algorithms become impractical above 10k dims.

Sparsity Problem

In 100D space, you'd need 2^{100} samples to fill it. You never have enough data — models see only empty space.

Model Complexity / Overfit

Too many irrelevant features add noise. The model learns noise instead of signal. Generalization collapses.

Benefits of Dimensionality Reduction: Faster training · Less storage · Better visualisation · Removes noise · Helps other algorithms (KNN, SVM, K-Means) work better

Feature Selection

Keep the most important features, drop the rest. Variance threshold, correlation filter, mutual information. Features stay interpretable.

Feature Extraction

Combine existing features into new ones. PCA, t-SNE, autoencoders. New features may not be directly interpretable but capture most information.

PCA — Principal Component Analysis

Find the directions of maximum variance in data and project onto them. The most widely used dim. reduction method.

What PCA does

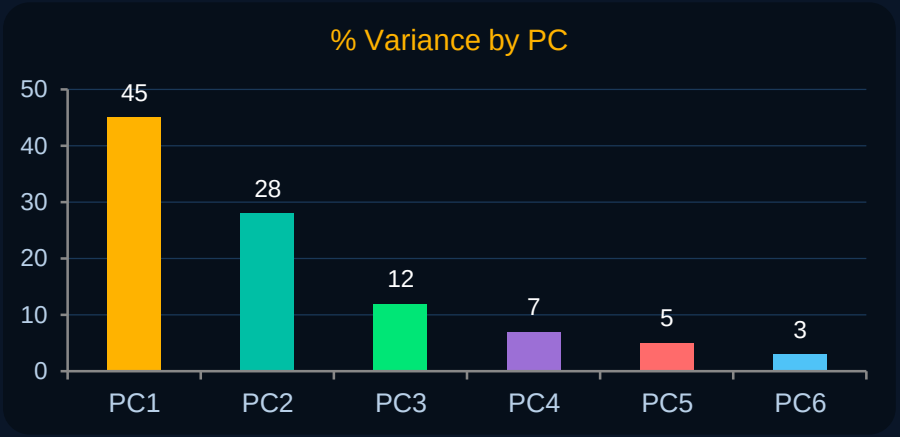
Rotates the coordinate system so the first axis (PC1) captures maximum variance, PC2 the next most, etc. It's a change of basis — no information is lost at this stage.

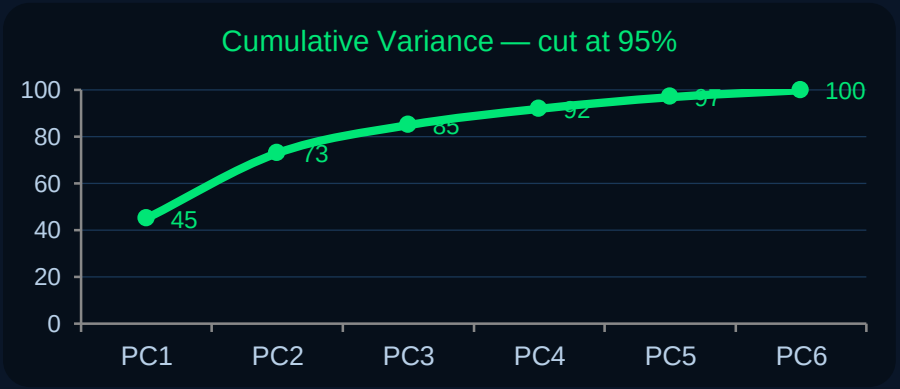
The Math

Compute covariance matrix → eigendecomposition → eigenvectors = principal components → eigenvalues = variance explained by each PC. Select top K.

How many PCs?

Plot cumulative explained variance vs number of components. Choose K where curve levels off (e.g., 95% variance). Use scree plot for visual guidance.





t-SNE & UMAP — Nonlinear Visualization Methods

When PCA's straight lines aren't enough — visualize complex manifolds in 2D/3D.

t-SNE — t-distributed Stochastic Neighbor Embedding

How it works	Converts distances to probabilities in high-D. Finds 2D layout where those probabilities match. Points that are similar → stay close in 2D.
Perplexity	Key parameter (5–50). Controls 'number of neighbors'. Lower = local structure. Higher = global structure. Tune carefully!
Non-parametric	No 'transform' for new data — you must re-run on the whole dataset. Can't transform a new test point.
Best for	Beautiful 2D/3D visualizations of clusters. Images, text embeddings, genomics data. Reveals structure invisible to PCA.
Weaknesses	Slow $O(n^2)$. No global structure guarantee. Random seed matters! Results may vary between runs. Can't be used for downstream modeling.

UMAP — Uniform Manifold Approximation and Projection

How it works	Based on Riemannian geometry and algebraic topology. Builds a weighted graph from high-D data and optimizes a low-D graph to match.
n_neighbors	Like perplexity in t-SNE. Controls local vs global structure balance. 15 is default. Try 5–100.
Parametric	Can transform new data points without rerunning! Great for production: fit on train, transform test.
Best for	Large datasets (faster than t-SNE). Preserves more global structure. Better for downstream tasks after dimensionality reduction.
vs t-SNE	5–10× faster. Preserves global structure better. More stable results. Also great for preprocessing before clustering or classification.

03

Anomaly

Detection

Anomaly Detection — Finding the Unusual

Identify data points that don't fit the normal pattern — no anomaly labels required.

Real examples: Credit card fraud · Network intrusion · Industrial sensor faults · Medical anomalies · Fake reviews · Rare disease patterns · Defective products on assembly line

Isolation Forest

How it works:

Build random trees that isolate points by random splits. Anomalies are isolated with fewer splits — they get shorter paths through the tree.

Key param: contamination param controls expected % of outliers.

Pros: Fast, scalable, works well in high dimensions, handles multivariate anomalies.

```
IsolationForest(contamination=0.05)
```

⚠ Anomaly score → threshold → flag

Local Outlier Factor (LOF)

How it works:

Compares the local density of a point to the density of its neighbors. Points in low-density neighborhoods relative to neighbors = outliers.

Key param: n_neighbors param: how many neighbors define 'local'.

Pros: Finds anomalies in varying-density data. Great for local outliers.

```
LocalOutlierFactor(n_neighbors=20)
```

⚠ Anomaly score → threshold → flag

One-Class SVM

How it works:

Learn a decision boundary around 'normal' training data in a high-D kernel space. New points outside the boundary = anomalies.

Key param: nu param controls the upper bound on outlier fraction.

Pros: Works well with clear normal boundary. Kernel trick handles nonlinear cases.

```
OneClassSVM(nu=0.05, kernel='rbf')
```

⚠ Anomaly score → threshold → flag

04

Association

Rules

Association Rules — Market Basket Analysis

Find items that frequently occur together in transactions. If {A} then → {B}.

Classic example: Walmart discovered that customers buying diapers also tend to buy beer on Friday evenings. They placed them close together and sales of BOTH increased. Data-driven insight — no human intuition needed.

Support

```
supp(A→B) = P(A ∪ B)
```

How often does the itemset {A,B} appear in ALL transactions? Measures frequency. Low support = rare pattern. Set min_support to filter.

Example: Out of 1000 baskets, {bread,butter} appears 200 times → support = 20%

Confidence

```
conf(A→B) = P(B|A) =
supp(A∪B)/supp(A)
```

Given A is in the basket, how likely is B also there? Measures rule reliability. High confidence = strong implication.

Example: Of 200 {bread} baskets, 140 also have butter → confidence = 70%

Lift

```
lift(A→B) = conf(A→B) / P(B)
```

How much more likely is B with A than without? Lift > 1 = positive association. Lift = 1 = independent. Lift < 1 = negative.

Example: Butter appears in 30% of all baskets. Lift = 70%/30% = 2.33 → strong!

Apriori Algorithm

Generate all candidate itemsets. Prune those below min_support. Join surviving ones to find larger sets. Simple but SLOW for large data — $O(2^n)$ candidates.

FP-Growth

Build a compact FP-tree structure. Mine patterns directly from the tree without generating candidates. 10–100× faster than Apriori. Preferred in practice.

Applications

Retail recommendations · Web page navigation patterns · Medical co-diagnosis patterns · Cross-selling · Playlist generation · Document co-occurrence

Unsupervised Learning in Python — Key Implementations

```python
# ── CLUSTERING ──────────────────────────
from sklearn.cluster import KMeans, DBSCAN
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# K-Means
km = KMeans(n_clusters=3, init='k-means++',
            n_init=10, random_state=42)
labels = km.fit_predict(X_scaled)
print('Inertia:', km.inertia_)

# DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
db_labels = db.fit_predict(X_scaled)  # -1=noise

# Silhouette score
from sklearn.metrics import silhouette_score
sil = silhouette_score(X_scaled, labels)
print(f'Silhouette: {sil:.3f}')

# ── DIMENSIONALITY REDUCTION ────────────
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

pca = PCA(n_components=0.95)  # keep 95% var
X_pca = pca.fit_transform(X_scaled)
print('Components:', pca.n_components_)
```

```python
# t-SNE for visualization
tsne = TSNE(n_components=2, perplexity=30,
            random_state=42)
X_2d = tsne.fit_transform(X_scaled)

# ── ANOMALY DETECTION ──────────────────
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor

iso = IsolationForest(contamination=0.05,
                      random_state=42)
scores = iso.fit_predict(X_scaled)
# scores: 1=normal, -1=anomaly

lof = LocalOutlierFactor(n_neighbors=20)
lof_scores = lof.fit_predict(X_scaled)

# ── ASSOCIATION RULES ──────────────────
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules

freq = apriori(df_encoded, min_support=0.2,
               use_colnames=True)
rules = association_rules(freq,
    metric='lift', min_threshold=1.5)
rules.sort_values('lift', ascending=False)
```

```
pip install scikit-learn mlxtend umap-learn  |  import umap; reducer = umap.UMAP(); X_umap = reducer.fit_transform(X_scaled)
```

Real-World Applications of Unsupervised Learning

E-Commerce

Customer Segmentation

K-Means groups customers by behavior → personalized marketing campaigns for each segment

→ K-Means

Streaming

Recommendation Systems

Find users with similar listening patterns (collaborative filtering uses clustering at core)

→ Hierarchical

Healthcare

Patient Sub-groups

Cluster patients with similar symptoms/genetics → personalized treatment protocols per group

→ DBSCAN + PCA

Cybersecurity

Intrusion Detection

Normal network traffic forms dense clusters. Attacks = rare anomalies with low density scores

→ Isolation Forest

NLP / Text

Topic Modeling (LDA)

Discover latent topics in thousands of documents without any topic labels — pure unsupervised

→ LDA / K-Means

Genomics

Gene Expression Analysis

Cluster thousands of genes or patients by expression profiles → discover disease subtypes

→ PCA + UMAP

Banking

Fraud Detection

Fraudulent transactions are statistically rare — anomaly detection catches them automatically

→ LOF + IF

Manufacturing

Predictive Maintenance

Sensor data from machines forms 'normal' clusters. Deviations predict failure before it happens

→ Autoencoder

Decision Guide — Which Method Should You Use?

	K-Means	DBSCAN	Hierarchical	PCA	t-SNE / UMAP	Isolation Forest
Goal: grouping	✓✓ Best	✓✓ Good	✓✓ Good	—	—	—
Goal: visualize	—	—	Dendrogram	✓✓ Linear	✓✓ Best 2D	—
Goal: anomalies	Partial	✓✓ Noise pts	—	—	—	✓✓ Best
Knows K?	Required	Not needed	After view	Choose PCs	Choose 2/3	—
Cluster shape	Sphere only	Any shape	Any shape	—	—	—
Large dataset	✓✓ Fast	Moderate	Slow $O(n^2)$	✓✓ Fast	Slow→UMAP	✓✓ Fast
Noisy data	Sensitive	Handles ✓✓	Sensitive	Partial	Partial	Designed ✓✓
Interpretability	High ✓✓	Medium	Very high ✓✓	Medium	Low	Medium

Workflow: Always start with EDA → PCA/UMAP for visualization → K-Means for quick baseline → DBSCAN if non-spherical → Isolation Forest for outliers → Hierarchical if you need interpretable hierarchy

Session Summary — Key Takeaways

01

No labels needed

Unsupervised learning discovers patterns from raw data. No annotations required — ideal for the 99% of unlabeled data in the real world.

02

Clustering groups similar data

K-Means (spherical, fast), DBSCAN (any shape, finds noise), Hierarchical (dendrogram, choose K later). Always scale features first!

03

Reduce before you model

High dimensions hurt everything. PCA = linear, fast, interpretable. t-SNE/UMAP = nonlinear visualization. Use PCA as preprocessing.

04

Anomaly = low density

Isolation Forest, LOF, and One-Class SVM detect outliers without ever seeing labeled anomalies. contamination sets expected rate.

05

Rules reveal co-occurrence

Association rules (Apriori/FP-Growth) find what items appear together. Support · Confidence · Lift — all three matter.

06

Evaluate without labels

Silhouette score, Davies-Bouldin index, inertia. Internal metrics tell you quality without needing ground truth labels.

Great work!

You now understand Unsupervised Learning.